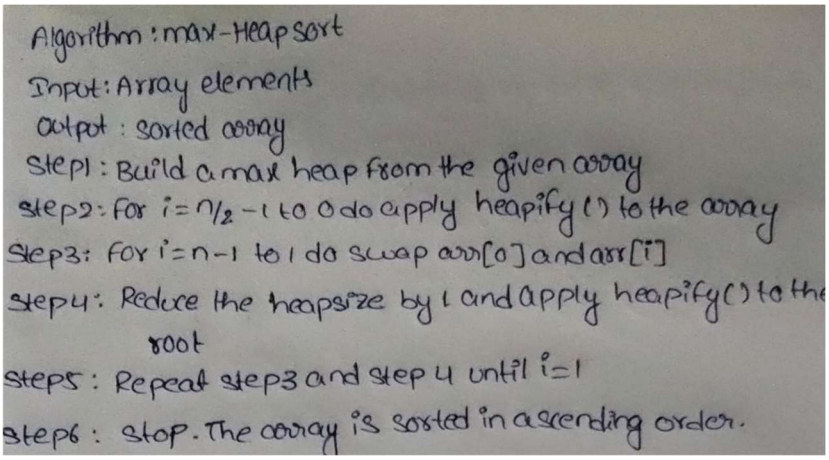
Practical – 3

Aim: Implimentation of max heap sort algorithm.

Algorithm:



Program:

```
#include <iostream>
using namespace std;

// Heapify function
void heapify(int arr[], int n, int i)
{
   int largest = i;
   int left = 2 * i + 1;
   int right = 2 * i + 2;

   // Check left child
   if (left < n && arr[left] > arr[largest])
      largest = left;

   // Check right child
   if (right < n && arr[right] > arr[largest])
      largest = right;

   // If largest is not root
   if (largest != i)
   {
      swap(arr[i], arr[largest]);
```

Name:B.Vennela
Enrollment no:92460118232

```
heapify(arr, n, largest);
    }
}

// Heap Sort
void heapSort(int arr[], int n)
{
    // Build Max Heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Remove elements from heap one by one
    for (int i = n - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);

        heapify(arr, i, 0);
    }
}

int main()
{
    int arr[] = {10, 5, 20, 2, 8};
    int n = 5;

    heapSort(arr, n);

    cout << "Sorted array: ";

    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }

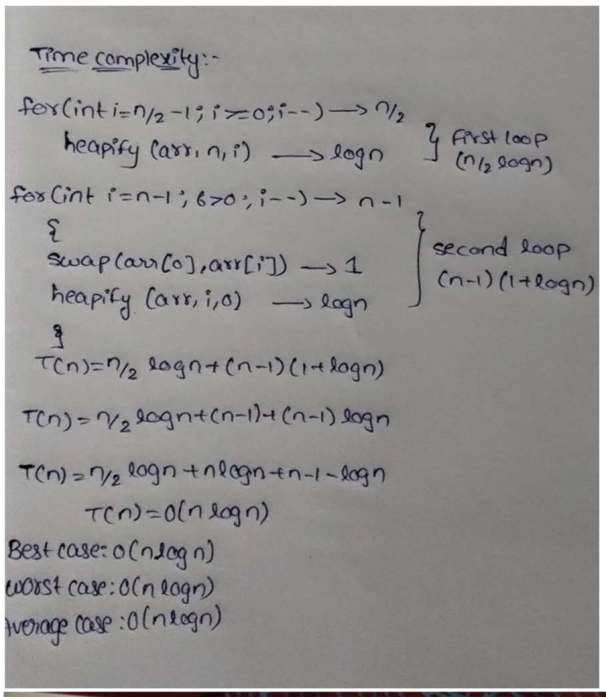
    return 0;
}
```

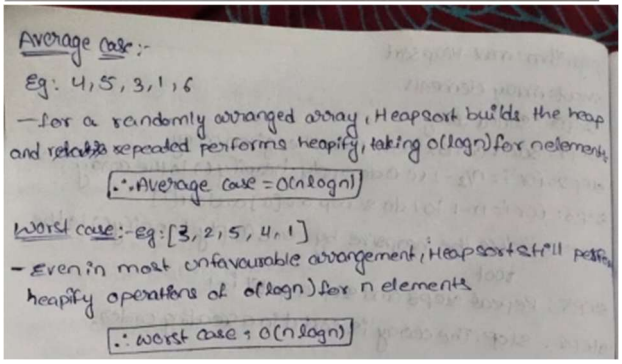
Output:

```
Original array: 12 11 13 5 6 7
Sorted array: 5 6 7 11 12 13


=== Code Execution Successful ===
```

Time complexity:





Name:B.Vennela
Enrollment no:92460118232

Conclusion:

Max-Heap Sort is an efficient sorting algorithm based on the Max Heap data structure. It repeatedly places the largest element at the end of the array. It has the a time complexity of O(n log n) and does not require extra memory for another array. Therefore, Max-Heap Sort is useful when we need an efficient and memory-saving sorting method.

Name:B.Vennela
Enrollment no:92460118232